4.7 In this exercise we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: `10101100011000100000000000010100`.

Decoding: opcode = `101011` (SW), rs = `00011` (r3), rt = `00010` (r2), imm = `0000000000010100` (20). So the instruction is `sw r2, 20(r3)`.

4.7.1 What are the outputs of the sign-extend and the jump "Shift left 2" unit for this instruction word?

- Sign-extend: `0x00000014` (= 20)
- Jump shift-left-2: take bits 25–0 (`00011000100000000000010100`) and shift left 2 → `0x01880050`

4.7.2 What are the values of the ALU control unit's inputs for this instruction?

ALUOp = `00` (SW needs add for address calc), funct field (bits 5–0) = `010100`. ALU control output = add.

4.7.3 What is the new PC address after this instruction is executed? Highlight the path through which this value is determined.

New PC = PC + 4 (SW doesn't branch/jump). Path: PC → PC+4 adder → PCSrc mux (selects PC+4 since branch=0) → Jump mux (selects PC+4 since Jump=0) → PC.

4.7.4 For each Mux, show the values of its data output during the execution of this instruction and these register values.

- PCSrc mux: PC+4
- Jump mux: PC+4
- RegDst mux: 2 (rt) — don't care, RegWrite=0
- ALUSrc mux: 20 (sign-ext immediate)
- MemtoReg mux: 0 — don't care, RegWrite=0

4.7.5 For the ALU and the two add units, what are their data input values?

- ALU: −3 (r3) and 20 (sign-ext imm) → output 17 (memory address)
- PC+4 adder: PC and 4
- Branch adder: PC+4 and 80 (= 20 « 2)

4.7.6 What are the values of all inputs for the "Registers" unit?

- Read reg 1 = 3, Read reg 2 = 2
- Write reg = 2 (from RegDst mux)
- Write data = 0 (don't care)
- RegWrite = 0

4.8.1 What is the clock cycle time in a pipelined and non-pipelined processor?

- Pipelined: max stage = 350 ps

- Non-pipelined: $250 + 350 + 150 + 300 + 200 = 1250$ ps

4.8.2 What is the total latency of an LW instruction in a pipelined and non-pipelined processor?

- Pipelined: $5 \times 350 = 1750$ ps
- Non-pipelined: 1250 ps

4.8.3 If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?

Split ID (350 ps) into two 175 ps stages. New cycle time = next-largest stage = 300 ps (MEM).

4.8.4 Assuming there are no stalls or hazards, what is the utilization of the data memory?

Only LW and SW touch data memory: $20\% + 15\% = 35\%$.

4.8.5 Assuming there are no stalls or hazards, what is the utilization of the write-register port of the "Registers" unit?

ALU and LW write back: $45\% + 20\% = 65\%$.

4.8.6 Compare clock cycle times and execution times with single-cycle, multi-cycle, and pipelined organization.

Cycles per instruction in multi-cycle: alu = 4, beq = 3, lw = 5, sw = 4.

Avg CPI $= 0.45(4) + 0.20(3) + 0.20(5) + 0.15(4) = 4.0$ cycles.

Org	Clock	Avg time/instr
Single-cycle	1250 ps	1250 ps
Multi-cycle	350 ps	$4.0 \times 350 = 1400$ ps
Pipelined	350 ps	350 ps (steady state)

Pipelined is fastest. Multi-cycle has shortest clock but ends up slower than single-cycle here because the ID stage dominates.

4.9.1 Indicate dependences and their type.

All RAW:

- I1 $\rightarrow$ I2 on r1
- I1 $\rightarrow$ I3 on r1
- I2 $\rightarrow$ I3 on r2

4.9.2 Assume there is no forwarding in this pipelined processor. Indicate hazards and add nop instructions to eliminate them.

Need 2 nops between dependent instructions (write in WB, read in ID, with same-cycle register file write/read):

```
or  r1,r2,r3
nop
```

```
nop
or  r2,r1,r4
nop
nop
or  r1,r1,r2
```

4.9.3 Assume there is full forwarding. Indicate hazards and add NOP instructions to eliminate them.

EX/MEM → EX covers I1→I2 and I2→I3; MEM/WB → EX covers I1→I3. No nops needed.

4.9.4 What is the total execution time of this instruction sequence without forwarding and with full forwarding? What is the speedup achieved by adding full forwarding to a pipeline that had no forwarding?

- No forwarding: 7 instr → $7 + 4 = 11$ cycles × 250 ps = 2750 ps
- Full forwarding: 3 instr → $3 + 4 = 7$ cycles × 300 ps = 2100 ps
- Speedup = $2750/2100 \approx 1.31$

4.9.5 Add nop instructions to this code to eliminate hazards if there is ALU-ALU forwarding only (no forwarding from the MEM to the EX stage).

All three are ALU ops, so EX/MEM→EX and MEM/WB→EX (ALU results) handle every dependence. No nops needed.

4.9.6 What is the total execution time of this instruction sequence with only ALU-ALU forwarding? What is the speedup over a no-forwarding pipeline?

7 cycles × 290 ps = 2030 ps. Speedup = $2750/2030 \approx 1.35$.